



آزمایشگاه طراحی سیستم‌های دیجیتال

ترم تابستان ۱۴۰۵

استاد: دکتر انصاری

آزمایش سوم

توصیف جریان داده (مقایسه‌کننده‌ی آبشاری و سریال)

خلاصه

در این آزمایش دو مدار مقایسه‌کننده با توصیف جریان داده (assign) در Verilog پیاده‌سازی شده‌اند و با Icarus Verilog (vvp/iverilog) صحت‌سنجی شده‌اند:

۱. یک Cascadable 1-bit comparator کاملاً ترکیبی با assign، و سپس یک مقایسه‌کننده‌ی چهاربیتی سلسله‌مراتبی ساخته‌شده از چهار نمونه‌ی آن.

۲. یک مقایسه‌کننده‌ی سریال ترتیبی که بیت‌به‌بیت (از پرارزش‌ترین بیت) دو عدد را می‌گیرد و در هر پالس ساعت نتیجه‌ی مقایسه‌ی تا همین‌جا را روی خروجی‌های LT/EQ/GT می‌گذارد. برای این قسمت دو نسخه نوشته شده: نسخه‌ی A (منطق next-state با assign + فلیپ‌فلاپ لبه‌حساس با always) و نسخه‌ی B (تمام ماژول فقط با assign، با زوج latch مستر-اسلیو برای نگهداری حالت).

قسمت ۱: مقایسه‌کننده‌ی ۱ بیتی آبشاری

سیگنال	جهت	پهنا	توضیح
B, A	ورودی	۱	بیت‌های همین طبقه
$LT_{in}, EQ_{in}, GT_{in}$	ورودی	۱	نتیجه‌ی cascade از بیت‌های پرارزش‌تر
$LT_{out}, EQ_{out}, GT_{out}$	خروجی	۱	نتیجه‌ی به‌روز تا این بیت

دانه اول (قبل از MSB) با $(GT, EQ, LT) = (0, 1, 0)$ تغذیه می‌شود؛ یعنی تا الان برابر. معادلات جریان داده:

$$EQ_{out} = EQ_{in} \wedge \neg(A \oplus B)$$

$$GT_{out} = GT_{in} \vee (EQ_{in} \wedge A \wedge \neg B)$$

$$LT_{out} = LT_{in} \vee (EQ_{in} \wedge \neg A \wedge B)$$

یعنی فقط وقتی هنوز برابر بوده‌ایم اجازه‌ی تصمیم جدید داریم؛ بعد از تصمیم GT یا LT، همان مقدار از طبقه‌های بعدی عبور می‌کند.

```

1 // Cascadable 1-bit comparator - pure dataflow (assign only).
2 // Cascade direction: MSB → LSB. Seed the MSB stage with (GT,EQ,LT)=(0,1,0).
3 module cascadable_1bit_comparator (
4     input wire A,
5     input wire B,
6     input wire GT_in,
7     input wire EQ_in,
8     input wire LT_in,
9     output wire GT_out,
10    output wire EQ_out,
11    output wire LT_out
12);
13    // Still equal only if already equal AND current bits match.
14    assign EQ_out = EQ_in & ~(A ^ B);
15
16    // Greater if already greater, OR still equal and A=1,B=0.
17    assign GT_out = GT_in | (EQ_in & A & ~B);
18
19    // Less if already less, OR still equal and A=0,B=1.

```

```
20 assign LT_out = LT_in | (EQ_in & ~A & B);
21 endmodule
```

قسمت ۱: مقایسه‌کننده ۴ بیتی سلسله‌مراتبی

چهار نمونه‌ی cascadable_1bit_comparator به زنجیره وصل شده‌اند:



مدار کاملاً ترکیبی است (بدون کلاک). خروجی نهایی از طبقه‌ی LSB برداشته می‌شود.

```

1 // 4-bit combinational comparator built hierarchically from four
2 // cascadable 1-bit stages (MSB → LSB). Pure structural + dataflow leaves.
3 module comparator_4bit (
4     input wire [3:0] A,
5     input wire [3:0] B,
6     output wire      GT,
7     output wire      EQ,
8     output wire      LT
9 );
10 wire gt3, eq3, lt3;
11 wire gt2, eq2, lt2;
12 wire gt1, eq1, lt1;
13
14 // Seed: "equal so far" before looking at any bit.
15 cascadable_1bit_comparator u3 (
16     .A(A[3]), .B(B[3]),
17     .GT_in(1'b0), .EQ_in(1'b1), .LT_in(1'b0),
18     .GT_out(gt3), .EQ_out(eq3), .LT_out(lt3)
19 );
20 cascadable_1bit_comparator u2 (
21     .A(A[2]), .B(B[2]),
22     .GT_in(gt3), .EQ_in(eq3), .LT_in(lt3),
23     .GT_out(gt2), .EQ_out(eq2), .LT_out(lt2)
24 );
25 cascadable_1bit_comparator u1 (
26     .A(A[1]), .B(B[1]),
27     .GT_in(gt2), .EQ_in(eq2), .LT_in(lt2),
28     .GT_out(gt1), .EQ_out(eq1), .LT_out(lt1)
29 );
30 cascadable_1bit_comparator u0 (
31     .A(A[0]), .B(B[0]),
32     .GT_in(gt1), .EQ_in(eq1), .LT_in(lt1),
33     .GT_out(GT), .EQ_out(EQ), .LT_out(LT)
34 );
35 endmodule
```

قسمت ۲: مقایسه‌کننده‌ی سریال

سیگنال	جهت	پهنا	توضیح
clk	ورودی	۱	کلاک
reset	ورودی	۱	ریست ناهم‌گام فعال‌بالا
A, B	ورودی	۱	بیت جاری (ترتیب MSB-اول)
LT, EQ, GT	خروجی	۱	نتیجه‌ی مقایسه تا بیت‌های دیده‌شده

وضعیت با دو بیت (eq_q, gt_q) کد می‌شود: بعد از ریست $(1, 0)$ یعنی EQ؛ $(0, 1)$ یعنی GT؛ $(0, 0)$ یعنی LT. معادلات next-state (برای هر دو نسخه یکسان):

$$eq_d = eq_q \wedge \neg(A \oplus B)$$

$$gt_d = gt_q \vee (eq_q \wedge A \wedge \neg B)$$

$$LT = \neg eq_q \wedge \neg gt_q$$

نسخه‌ی A: serial_comparator_ff

منطق بالا با assign، و دو فلیپ‌فلاپ با @(posedge clk or posedge reset) always. خروجی‌ها روی لبه‌ی بالارونده به‌روز می‌شوند.

```

1 // Serial bit-by-bit comparator - approach A.
2 // Next-state logic is pure dataflow (assign); state is held in edge-
3 // triggered flip-flops (always @(posedge clk or posedge reset)).
4 // Bits arrive MSB-first on A/B each clock; outputs reflect the comparison
5 // of all bits seen so far (including the bit sampled on this edge).
6 module serial_comparator_ff (
7     input wire clk,
8     input wire reset,    // async active-high
9     input wire A,
10    input wire B,
11    output wire GT,
12    output wire EQ,
13    output wire LT
14);
15 // State: eq_q=1 means "equal so far"; gt_q=1 means "A>B decided".
16 // Encoding after reset: (eq_q, gt_q) = (1, 0) → EQ
17 //                      (0, 1) → GT
18 //                      (0, 0) → LT
19 reg eq_q, gt_q;
20
21 wire eq_d, gt_d;
22
23 // Next-state (combinational dataflow):
24 // Stay equal only while still equal AND current bits match.
25 assign eq_d = eq_q & ~(A ^ B);
26 // Become/stay greater if already greater, OR still equal and A>B this bit.
27 assign gt_d = gt_q | (eq_q & A & ~B);
28 // LT is implied: ~eq_d & ~gt_d
29
30 always @(posedge clk or posedge reset) begin
31     if (reset) begin
32         eq_q <= 1'b1;
33         gt_q <= 1'b0;
34     end else begin
35         eq_q <= eq_d;
36         gt_q <= gt_d;
37     end
38 end
39
40 assign EQ = eq_q;
41 assign GT = gt_q;
42 assign LT = ~eq_q & ~gt_q;
43 endmodule

```

نسخه‌ی B: serial_comparator_assign

طبق متن آزمایش، کل ماژول فقط assign است (بدون always و بدون نمونه‌سازی زیرماژول). نگهداری حالت با زوج latch مستر-اسلیو:

- وقتی $\text{clk} = 1$: مستر next-state را نمونه‌برداری می‌کند.
- وقتی $\text{clk} = 0$: اسلیو از مستر به‌روز می‌شود (خروجی‌ها اینجا عوض می‌شوند).

این رفتار معادل یک فلیپ‌فلاپ حساس به لبه‌ی پایین‌رونده است؛ در تست‌بنچ بیت‌ها هنگام $\text{clk} = 0$ آماده می‌شوند، سپس یک سیکل کامل $0 \rightarrow 1$ زده می‌شود و خروجی بعد از افت کلاک خوانده می‌شود — همان قرارداد یک بیت در هر پالس برای هر دو نسخه.

```

1 // Serial bit-by-bit comparator - approach B.
2 // Entire module is continuous-assign only (no always, no hierarchical
3 // instances). State is held by a -masterslave latch pair built from
4 // assign feedback, which behaves like a negative-edge-triggered FF:
5 // * clk=1 → master samples next-state
6 // * clk=0 → slave updates from master (outputs change here)
7 // Testbenches drive A/B while clk=1 and sample outputs after the falling
8 // edge, matching the FF version's "one bit per clock cycle" contract.
9 module serial_comparator_assign (
10     input wire clk,
11     input wire reset,    // async active-high (overrides latches)
12     input wire A,

```

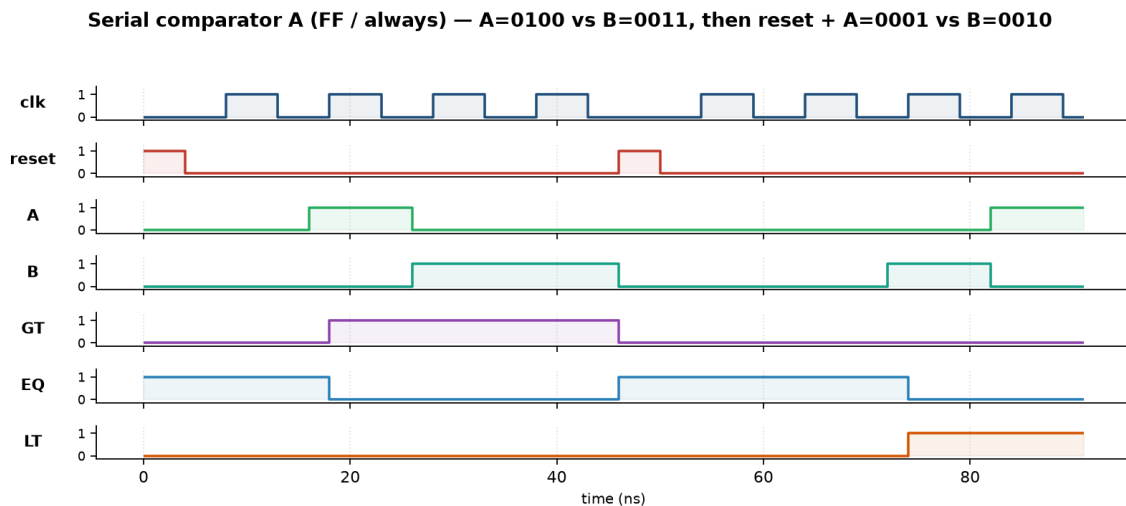
```

13 input wire B,
14 output wire GT,
15 output wire EQ,
16 output wire LT
17 );
18 wire nclk;
19 assign nclk = ~clk;
20
21 // Slave outputs (= current state visible to next-state logic & pins).
22 wire eq_q, gt_q;
23
24 // Next-state (same equations as approach A).
25 wire eq_d, gt_d;
26 assign eq_d = eq_q & ~(A ^ B);
27 assign gt_d = gt_q | (eq_q & A & ~B);
28
29 // Master latches (transparent while clk=1).
30 wire eq_m, gt_m;
31 assign eq_m = reset ? 1'b1 : (clk ? eq_d : eq_m);
32 assign gt_m = reset ? 1'b0 : (clk ? gt_d : gt_m);
33
34 // Slave latches (transparent while clk=0) - capture on falling edge.
35 assign eq_q = reset ? 1'b1 : (nclk ? eq_m : eq_q);
36 assign gt_q = reset ? 1'b0 : (nclk ? gt_m : gt_q);
37
38 assign EQ = eq_q;
39 assign GT = gt_q;
40 assign LT = ~eq_q & ~gt_q;
41 endmodule

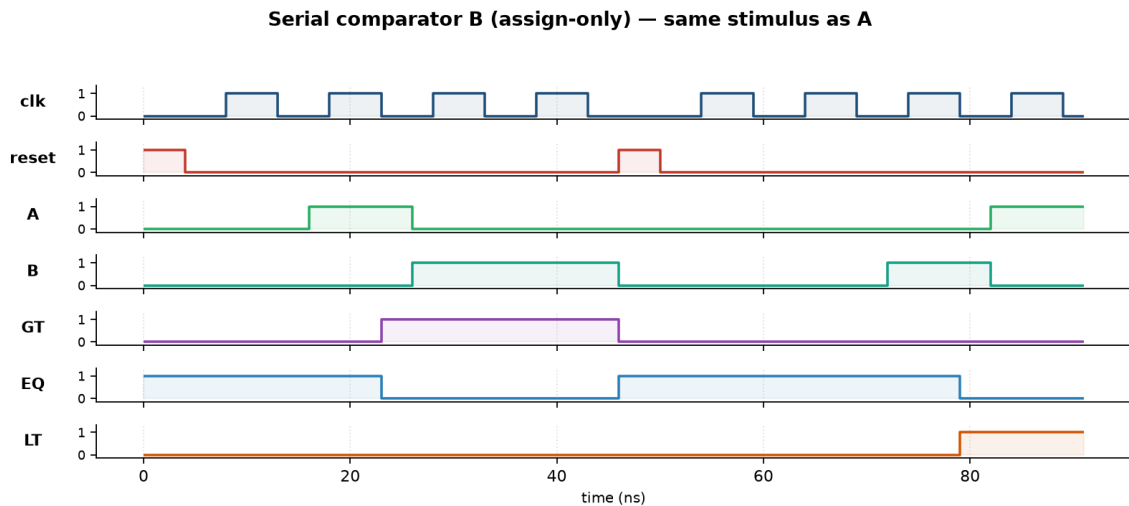
```

شکل موج

هر دو شکل موج از یک تست‌بنچ مشترک (tb_serial_wave) با همان دنباله گرفته شده‌اند: ابتدا $A=0100$ در برابر $B=0011$ ، سپس ریست وسط‌راه، سپس $A=0001$ در برابر $B=0010$. تفاوت ظاهری فقط در لبه به‌روزرسانی خروجی است: نسخه A روی لبه بالارونده، نسخه B روی لبه پائین‌رونده.



شکل ۱ - نسخه A (serial_comparator_ff): فلیپ‌فلاپ با @posedge clk



شکل ۲ - نسخه B (serial_comparator_assign): فقط assign (مستتر-اسلیو)

تست‌بنچ‌ها

تست‌بنچ‌های tb_comparator_4bit و tb_cascadable_1bit تمامی فضای ورودی را exhaustive چک می‌کنند. تست‌بنچ سریال (tb_serial_comparator) با ماکروی DUT_MODULE روی هر دو نسخه A و B اجرا می‌شود:

```

1 `timescale 1ns/1ps
2
3 module tb_cascadable_1bit;
4     reg A, B, GT_in, EQ_in, LT_in;
5     wire GT_out, EQ_out, LT_out;
6
7     cascadable_1bit_comparator dut (
8         .A(A), .B(B),
9         .GT_in(GT_in), .EQ_in(EQ_in), .LT_in(LT_in),
10        .GT_out(GT_out), .EQ_out(EQ_out), .LT_out(LT_out)
11    );
12
13    integer i, pass, fail;
14    reg exp_gt, exp_eq, exp_lt;
15
16    initial begin
17        pass = 0;
18        fail = 0;
19        for (i = 0; i < 32; i = i + 1) begin
20            {A, B, GT_in, EQ_in, LT_in} = i[4:0];
21            #1;
22
23            exp_eq = EQ_in & ~(A ^ B);
24            exp_gt = GT_in | (EQ_in & A & ~B);
25            exp_lt = LT_in | (EQ_in & ~A & B);
26
27            if (GT_out === exp_gt && EQ_out === exp_eq && LT_out === exp_lt)
28                pass = pass + 1;
29            else begin
30                $display("FAIL i=%0d A=%b B=%b in(G,E,L)=%b%b%b got=%b%b%b exp=%b%b%b",
31                    i, A, B, GT_in, EQ_in, LT_in,
32                    GT_out, EQ_out, LT_out, exp_gt, exp_eq, exp_lt);
33                fail = fail + 1;
34            end
35        end
36        $display("tb_cascadable_1bit: Passed: %0d, Failed: %0d", pass, fail);
37        if (fail != 0) $fatal(1);
38        $finish;
39    end
40 endmodule

```

```

1 `timescale 1ns/1ps
2
3 module tb_comparator_4bit;
4     reg [3:0] A, B;
5     wire      GT, EQ, LT;
6
7     comparator_4bit dut (.A(A), .B(B), .GT(GT), .EQ(EQ), .LT(LT));

```

```

8
9 integer ai, bi, pass, fail;
10 reg exp_gt, exp_eq, exp_lt;
11
12 initial begin
13     pass = 0;
14     fail = 0;
15     for (ai = 0; ai < 16; ai = ai + 1) begin
16         for (bi = 0; bi < 16; bi = bi + 1) begin
17             A = ai[3:0];
18             B = bi[3:0];
19             #1;
20             exp_gt = (ai > bi);
21             exp_eq = (ai == bi);
22             exp_lt = (ai < bi);
23             if (GT == exp_gt && EQ == exp_eq && LT == exp_lt)
24                 pass = pass + 1;
25             else begin
26                 $display("FAIL A=%0d B=%0d got GEL=%b%b%b exp=%b%b%b",
27                     ai, bi, GT, EQ, LT, exp_gt, exp_eq, exp_lt);
28                 fail = fail + 1;
29             end
30         end
31     end
32     $display("tb_comparator_4bit: Passed: %0d, Failed: %0d", pass, fail);
33     if (fail != 0) $fatal(1);
34     $finish;
35 end
36 endmodule

```

```

1 `timescale 1ns/1ps
2
3 // Shared stimulus for both serial comparator variants.
4 // Compile with -DDUT_MODULE=serial_comparator_ff (default)
5 // or -DDUT_MODULE=serial_comparator_assign
6 `ifndef DUT_MODULE
7 `define DUT_MODULE serial_comparator_ff
8 `endif
9
10 module tb_serial_comparator;
11     reg clk, reset, A, B;
12     wire GT, EQ, LT;
13
14     `DUT_MODULE dut (
15         .clk(clk), .reset(reset), .A(A), .B(B),
16         .GT(GT), .EQ(EQ), .LT(LT)
17     );
18
19     integer pass, fail;
20
21     // Present A/B while clk=0, raise clk (FF samples / master opens),
22     // then drop clk (assign slave updates). Sample after falling edge.
23     task automatic cycle;
24         input bit_a;
25         input bit_b;
26         begin
27             A = bit_a;
28             B = bit_b;
29             #2;
30             clk = 1;
31             #5;
32             clk = 0;
33             #3;
34         end
35     endtask
36
37     task automatic do_reset;
38         begin
39             clk = 0;
40             A = 0;
41             B = 0;
42             reset = 1;
43             #4;
44             reset = 0;
45             #2;
46         end
47     endtask
48
49     task automatic check;
50         input exp_gt;
51         input exp_eq;
52         input exp_lt;
53         input [255:0] tag;
54         begin
55             if (GT == exp_gt && EQ == exp_eq && LT == exp_lt) begin
56                 pass = pass + 1;

```

```

57         end else begin
58             $display("FAIL [%0s] got GEL=%b%b%b exp=%b%b%b",
59                 tag, GT, EQ, LT, exp_gt, exp_eq, exp_lt);
60             fail = fail + 1;
61         end
62     end
63 endtask
64
65 // Feed 4 bits MSB-first; check after each bit.
66 // Prefix va[3:k] == (va >> k) as an unsigned integer comparison.
67 task automatic feed4_check;
68     input [3:0] va;
69     input [3:0] vb;
70     input [255:0] tag;
71     integer k;
72     integer rem_a, rem_b;
73     begin
74         do_reset;
75         check(1'b0, 1'b1, 1'b0, {tag, "/reset"});
76         for (k = 3; k >= 0; k = k - 1) begin
77             cycle(va[k], vb[k]);
78             rem_a = va >> k;
79             rem_b = vb >> k;
80             check(rem_a > rem_b, rem_a == rem_b, rem_a < rem_b, tag);
81         end
82     end
83 endtask
84
85 initial begin
86     pass = 0;
87     fail = 0;
88     clk = 0;
89     reset = 0;
90     A = 0;
91     B = 0;
92
93     feed4_check(4'b0000, 4'b0000, "eq_0");
94     feed4_check(4'b1010, 4'b1010, "eq_A");
95     feed4_check(4'b1111, 4'b1111, "eq_F");
96
97     feed4_check(4'b1000, 4'b0111, "gt_msb");
98     feed4_check(4'b0100, 4'b0011, "gt_bit2");
99     feed4_check(4'b0010, 4'b0001, "gt_bit1");
100    feed4_check(4'b0001, 4'b0000, "gt_lsb");
101
102    feed4_check(4'b0111, 4'b1000, "lt_msb");
103    feed4_check(4'b0011, 4'b0100, "lt_bit2");
104    feed4_check(4'b0000, 4'b0001, "lt_lsb");
105
106    // Mid-stream reset then sticky LT decision on 0001 vs 0010
107    do_reset;
108    cycle(1'b1, 1'b0);
109    check(1'b1, 1'b0, 1'b0, "mid_before_rst");
110    do_reset;
111    check(1'b0, 1'b1, 1'b0, "mid_after_rst");
112    cycle(1'b0, 1'b0); check(1'b0, 1'b1, 1'b0, "reentry1");
113    cycle(1'b0, 1'b0); check(1'b0, 1'b1, 1'b0, "reentry2");
114    cycle(1'b0, 1'b1); check(1'b0, 1'b0, 1'b1, "reentry3");
115    cycle(1'b1, 1'b0); check(1'b0, 1'b0, 1'b1, "reentry4_sticky");
116
117    // Exhaustive 4-bit pairs (final result)
118    begin : exhaustive
119        integer ai, bi, k;
120        reg [3:0] va, vb;
121        for (ai = 0; ai < 16; ai = ai + 1) begin
122            for (bi = 0; bi < 16; bi = bi + 1) begin
123                va = ai[3:0];
124                vb = bi[3:0];
125                do_reset;
126                for (k = 3; k >= 0; k = k - 1)
127                    cycle(va[k], vb[k]);
128                check(ai > bi, ai == bi, ai < bi, "exh");
129            end
130        end
131    end
132
133    $display("tb_serial: Passed: %0d, Failed: %0d", pass, fail);
134    if (fail != 0) $fatal(1);
135    $finish;
136 end
137 endmodule

```

نتایج تست

همه‌ی شبیه‌سازی‌ها با Icarus Verilog 12.0:

نتیجه	پوشش	تست
۳۲/۳۲	همه‌ی ۳۲ ترکیب ورودی ۵ بیتی	tb_cascadable_1bit
۲۵۶/۲۵۶	همه‌ی ۲۵۶ جفت $A, B \in \{0..15\}$	tb_comparator_4bit
۳۱۲/۳۱۲	نمونه‌ها + ریست وسط‌راه + exhaustive نهایی	tb_serial (نسخه‌ی A)
۳۱۲/۳۱۲	همان بردارها روی ماژول فقط-assign	tb_serial (نسخه‌ی B)

خروجی واقعی run_tests.sh:

```
iverilog: Icarus Verilog version 12.0 (stable) ()

=== tb_cascadable_1bit ===
tb_cascadable_1bit: Passed: 32, Failed: 0
PASS: tb_cascadable_1bit

=== tb_comparator_4bit ===
tb_comparator_4bit: Passed: 256, Failed: 0
PASS: tb_comparator_4bit

=== tb_serial_ff ===
tb_serial: Passed: 312, Failed: 0
PASS: tb_serial_ff

=== tb_serial_assign ===
tb_serial: Passed: 312, Failed: 0
PASS: tb_serial_assign

summary: 4/4 passed, 0 failed
```